VIZZIO Deployment Platform

Technology Stack Proposal

Field	Details
Project	VIZZIO Deployment Platform — 8-Week Internship
Document Type	Technology Stack Proposal
Version	v1.0
Prepared By	Internship Team (CHAY, Vearak & NHEN, Sophamony)
Date	28 May 2025

I. System Overview

The Vizzio Deployment Platform is a secure software distribution system designed to deliver large Unreal Engine builds to authorized users. The system follows a client-server architecture and includes web-based interfaces for administration and user access, along with a native Windows application for build delivery and installation.

The platform is composed of the following components:

- Admin Web Application: system management and deployment control
- User Web Application: client access to available builds and updates
- Backend API: authentication, business logic, and system operations
- File Delivery Service: high-performance distribution of large build files
- Windows Desktop Launcher: local installation and update management

II. Technology Stack

a. Web Frontend (Admin & User Panels)

Technology: React + Vite

Both admin and user interfaces are built using a modern single-page application (SPA) architecture.

Admin Panel Responsibilities:

- Manage users and permissions
- Create and manage deployments
- Upload and organize build versions
- Monitor system activity

User Panel Responsibilities:

- Authenticate users
- Display available builds and versions

- Provide download access via launcher integration
- Show deployment history and updates

Rationale:

- Fast development and build performance
- Lightweight and scalable SPA architecture
- Easy deployment as static assets via Nginx
- Strong ecosystem and developer familiarity

b. Backend API

Technology: Node.js + Express

The backend provides core system functionality and exposes RESTful APIs for all client applications.

Responsibilities:

- User authentication and authorization
- Deployment and version management
- Generation of secure download tokens
- Business logic processing
- Communication with PostgreSQL database

Rationale:

- Lightweight and efficient API development
- Strong JavaScript ecosystem
- Rapid development suitable for internship timeline
- Easy integration with frontend and database layers

c. Database

Technology: PostgreSQL

The system uses a relational database to manage structured and interconnected data.

Stored Data:

- Users and roles
- Deployment projects
- Build versions
- Access permissions
- Download logs

Rationale:

- Strong relational data integrity
- ACID compliance for reliability

- Efficient handling of permission-based systems
- Mature tooling for backup and scaling

d. File Delivery System

Technology: Nginx

Nginx is used as a dedicated high-performance file server for Unreal Engine builds.

Responsibilities:

- Direct delivery of large build files
- Support for HTTP range requests (resumable downloads)
- High-throughput static file serving
- Offloading traffic from backend API

Rationale:

- Optimized for large file streaming
- Low CPU and memory overhead
- Improves scalability and system stability
- Industry-standard approach for file distribution

e. Windows Desktop Launcher

Technology: C# .NET 8 (WPF)

The launcher is a native Windows application installed on client machines.

Responsibilities:

- Authenticate users securely
- Download and install builds
- Verify file integrity (SHA-256 checks)
- Resume interrupted downloads
- Manage local installation directories
- Auto-update installed builds

Rationale:

- Native Windows performance and system access
- Efficient memory usage compared to Electron-based solutions
- Strong support for file system and networking operations
- Seamless integration with Windows security features

f. Installer Packaging

Technology: NSIS / Inno Setup

Used to package and distribute the Windows launcher.

Rationale:

- Lightweight installer generation
- Industry-standard Windows deployment tools
- Supports versioning and upgrade workflows

III. Security Architecture

The platform implements multiple layers of security:

- **Password Security:** bcrypt hashing for secure credential storage
- **Authentication:** JWT-based stateless authentication system
- **Download Security:** Short-lived signed tokens for file access
- **Token Validation:** Backend verification integrated with Nginx (auth_request)
- **Rate Limiting:** Protection on authentication endpoints
- **Credential Storage:** Secure storage using Windows Credential Manager

IV. System Architecture

a. High-Level Architecture

- React Web Applications (Admin + User) communicate with Backend API
- Backend API interacts with PostgreSQL database
- Nginx handles static file delivery and large build distribution
- Windows Launcher communicates with Backend and Nginx for downloads

b. Design Principles

- Separation of concerns between frontend, backend, and file delivery
- Stateless authentication using JWT
- Independent scaling of file delivery layer
- Minimal backend load for large file transfers
- Modular architecture for future expansion

V. Deployment Environment

The system is designed for deployment on a single Linux server environment:

Base System:

- Ubuntu Server 22.04 LTS

Services:

- Nginx
- Node.js (Backend API)
- PostgreSQL

Deployment Options:

- Cloud infrastructure (AWS, DigitalOcean, Hetzner)
- On-premise servers

The architecture is platform-independent and can be migrated without code modification.

VI. Conclusion

The selected technology stack prioritizes development speed, system stability, and efficient distribution of large-scale Unreal Engine builds while remaining suitable for long-term expansion.